Review Lab

CORE 103 Computational Thinking

Fall 2026

Instructions

- This Review Lab is **GRADED**.
- This Review Lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *ReviewLab_aa123456.zip* where *aa123456* will be replaced with your student id.

Exercises

1. Hostel Photo Wall Clips

This question should be implemented in the file `q1.py`.

A university student is decorating their hostel room with a photo string containing pictures of friends, family, and campus memories. The string is L centimeters long.

Photo clips are attached every D centimeters, starting exactly at position 0 and ending exactly at position L. The length of the string is chosen so that the clips fit evenly, with no position skipped.

Given L and D, determine the total number of photo clips required.

Constraints

- $L, D \in \mathbb{Z}$
- $1 \leq D \leq L \leq 1000$
- L is divisible by D

Interaction

The input comprises a single line containing 2 space-separated integers denoting the values of L and D respectively.
The output must contain a single number denoting the total number of photo clips required.

Sample

Input	Output
24 6	5
35 7	6

In the first case, $(L, D) = (24, 6)$. The clips are placed at positions 0, 6, 12, 18, and 24. Therefore, a total of 5 photo clips are required.

In the second case, $(L, D) = (35, 7)$. The clips are placed at positions 0, 7, 14, 21, 28, and 35. Therefore, a total of 6 photo clips are required.

Exercise
Indicate the outputs for the given inputs in `q1.py`.

Input	Output
18 3	
48 8	
63 9	

Pseudocode
Implement the pseudocode in `q1.py`.

2. Paired Programming Partner

This question should be implemented in the file `q2.py`.

A programming class is arranging students for a paired activity. There are P pairs in total, so the roll numbers go from 1 to $2P$.

The instructor pairs students from opposite ends of the roll-number list: roll number 1 with roll number $2P$, roll number 2 with roll number $2P - 1$, and so on.

Given your roll number R, determine your partner's roll number.

Constraints

- $P, R \in \mathbb{Z}$
- $1 \leq P \leq 300$
- $1 \leq R \leq 2P$

Interaction
The input comprises a single line containing 2 space-separated integers denoting the values of P and R, respectively.
The output must contain a single number denoting the roll number of your partner.

Sample

Input	Output
2 2	3
3 1	6

In the first case, $(P, R) = (2, 2)$. There are $2 \cdot 2 = 4$ students in the class and your roll number is 2. The pairs will be as follows: $\{(1, 4), (2, 3)\}$. You are partnered with roll number 3.

In the second case, $(P, R) = (3, 1)$. There are $2 \cdot 3 = 6$ students in the class and your roll number is 1. The pairs will be as follows: $\{(1, 6), (2, 5), (3, 4)\}$. You are partnered with roll number 6.

Exercise
Indicate the outputs for the given inputs in `q2.py`.

Input	Output
15 5	
35 23	
123 189	

Pseudocode
Implement the pseudocode in `q2.py`.

3. Part-Time Pay Calculator

This question should be implemented in the file `q3.py`.

A university student takes paid event shifts for a campus event company. The company calculates the event stipend using a base amount plus two support amounts. The support rule is different for smaller and larger event contracts.

For smaller event contracts below Rs. 1500, the travel support is 10% of the base amount and the event bonus is 90% of the base amount. Otherwise, the travel support is fixed at Rs. 500 and the event bonus is 98% of the base amount. Given s, calculate the total stipend.

Constraints

- $s \in \mathbb{N}$
- $500 \leq s \leq 50000$

Interaction
The input contains a single integer: s, the base amount.
The output must contain a float denoting the total stipend.

Sample

Input	Output
1203	2406.0
10042	20383.16

In the first case, $s = 1203$, which is less than 1500. The travel support is 120.3 and the event bonus is 1082.7. The total stipend is $1203 + 120.3 + 1082.7 = 2406.0$.

In the second case, $s = 10042$, which is at least 1500. The travel support is 500 and the event bonus is 9841.16. The total stipend is $10042 + 500 + 9841.16 = 20383.16$.

Exercise
Indicate the outputs for the given inputs in `q3.py`.

Input	Output
1312	
2300	
1899	

Python Code
Implement the Python code in `q3.py`.

4. Campus Receipt Code

This question should be implemented in the file `q4.py`.

A group of students is ordering snacks from the campus cafe for a small birthday surprise. The cafe billing system prints a special receipt code only when the final bill amount falls in the five-digit range.

The group buys n snack boxes, and each box costs p rupees. Decide whether the final bill amount can be printed as a valid receipt code. A valid amount is between 10000 and 99999 inclusive.

Constraints

- $n, p \in \mathbb{N}$
- $1 \leq n \leq 500$
- $1 \leq p \leq 1000$

Interaction
The input contains two space-separated integers: n and p.
The output must be "YES" if the total bill is a valid 5-digit receipt code, and "NO" otherwise.

Sample

Input	Output
25 785	YES
402 11	NO

In the first case, $n = 25$ and $p = 785$. The total bill is $25 \times 785 = 19625$, which is a valid 5-digit code, so the output is "YES".

In the second case, $n = 402$ and $p = 11$. The total bill is $402 \times 11 = 4422$, which is not a 5-digit code, so the output is "NO".

Exercise
Indicate the outputs for the given inputs in `q4.py`.

Input	Output
100 100	
33 12	
130 120	

Python Code
Implement the Python code in `q4.py`.

5. Family Trip Buffet Budget

This question should be implemented in the file `q5.py`.

Areeba's family is staying at a hill resort for a weekend trip. Breakfast, lunch, and dinner buffets are charged separately for each guest.

The resort charges different buffet prices for adults, children, and toddlers. Since every family member has different food plans for the day, Areeba wants a small function that can calculate the daily buffet bill for any one person.

The buffet charges are as follows:

	Breakfast	Lunch	Dinner
Adult (ages 13 and above)	Rs. 800	Rs. 1200	Rs. 1500
Child (ages 6 to 12)	Rs. 400	Rs. 700	Rs. 800
Toddler (ages 5 and below)	FREE	FREE	Rs. 300

For one family member, the input gives their age and three buffet choices: `breakfast_choice`, `lunch_choice`, `dinner_choice`. Each choice is 'Y' or 'y' if the person wants that buffet, and 'N' or 'n' otherwise.

Task
Write a function, `daily_buffet_cost`, that takes the following parameters:

- `age`
- `breakfast_choice`
- `lunch_choice`
- `dinner_choice`

The function must calculate the total daily buffet charges for that person and print the appropriate message using *Adult*, *Child*, or *Toddler*, according to the person's age.

After defining the function, read the four input values and call the function.

Constraints

- `age` is an integer.
- `age` > 0
- `breakfast_choice`, `lunch_choice`, and `dinner_choice` are each one of 'Y', 'y', 'N', or 'n'.

Interaction
The input contains four lines: `age`, `breakfast_choice`, `lunch_choice`, `dinner_choice`.
The program must print one statement giving the total daily buffet charges for the guest, using *Adult*, *Child*, or *Toddler* according to the person's age, as shown in the Sample section.

Sample

Input	Output
28 / Y / y / N	Adult charges for the day will be: Rs. 2000
10 / y / n / N	Child charges for the day will be: Rs. 400
5 / y / y / y	Toddler charges for the day will be: Rs. 300

In the first case, the guest is 28 years old, so the guest is an adult. The guest selects breakfast and lunch only. The total charges are Rs. $800 + 1200 = 2000$.

In the second case, the guest is 10 years old, so the guest is a child. The guest selects only breakfast. The total charges are Rs. 400.

In the third case, the guest is 5 years old, so the guest is a toddler. Breakfast and lunch are free for toddlers, while dinner costs Rs. 300. Therefore, the total charges are Rs. 300.

Python Implementation
Implement the Python code in `q5.py`.

6. Exchange Student Clock Helper

This question should be implemented in the file `q6.py`.

An exchange student has just arrived on campus and keeps mixing up the 24-hour clock used in schedules with the 12-hour format they are used to. They have already missed one society meeting because 13:05 looked confusing.

To help, you are asked to create a function that converts a given 24-hour time into the corresponding 12-hour AM/PM format.

Task
Write a function, `time_conversion`, that takes `hour_24` and `minute` as parameters.

If the given time is valid, the function must return the corresponding 12-hour time as a string followed by "AM" or "PM".

The minute part must always contain two digits. For example, 5 minutes should appear as 05.

Using a Built-in Function
For this question, you may use Python's `str()` function and string concatenation with `+` when building the returned time string. For example:

```
>>> minute = 5
>>> "0" + str(minute)
'05'
>>> hour_12 = 1
>>> str(hour_12) + ":05 PM"
'1:05 PM'
```

If either `hour_24` or `minute` is outside the valid range, the function must return "Invalid Time".

After defining the function, read the two input values, call the function, and print the returned value.

Constraints

- `hour_24` is an integer.
- `minute` is an integer.
- The input values may represent either a valid or an invalid time.

A valid 24-hour time satisfies:

- $0 \leq$ `hour_24` ≤ 23
- $0 \leq$ `minute` ≤ 59

Interaction
The input contains two lines: `hour_24`, `minute`.
The program must print the converted time in 12-hour AM/PM format if the input is valid, or "Invalid Time" otherwise.

Sample

Input	Output
13 / 5	1:05 PM
12 / 61	Invalid Time
0 / 23	12:23 AM
12 / 0	12:00 PM

In the first case, 13:05 in 24-hour format corresponds to 1:05 PM.

In the second case, 61 is not a valid value for minutes, so the function returns "Invalid Time".

In the third case, 00:23 corresponds to 12:23 AM.

In the fourth case, 12:00 corresponds to 12:00 PM.

Python Implementation
Implement the Python code in `q6.py`.

7. Triangular Number Sequence

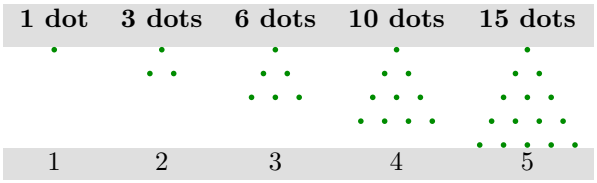
This question should be implemented in the file `q7.py`.

The triangular number sequence begins as follows:

$$1, 3, 6, 10, 15, 21, 28, 36, 45, \ldots$$

Each term represents the number of dots that can be arranged to form a triangle. The first term contains 1 dot, the second term contains 3 dots, the third term contains 6 dots, and so on.

Each new term can be obtained by adding the next positive integer to the previous term.



Function Specification
Write an iterative function named `triangle_number_sequence` that takes one integer parameter, `term_count`. The function must print the first `term_count` terms of the triangular number sequence.

The entire sequence must be printed on one line, with consecutive terms separated by a comma followed by one space. Do not print a trailing comma after the final term.

Constraints

- `term_count` is an integer.
- $1 \leq$ `term_count` ≤ 20

Sample Interaction

```
>>> triangle_number_sequence(1)
1
>>> triangle_number_sequence(4)
1, 3, 6, 10
```

For `term_count` = 1, only the first term of the sequence is printed.

For `term_count` = 4, the first four triangular numbers are 1, 3, 6, and 10, so the function prints:

```
1, 3, 6, 10
```

Output Formatting
Python's `print()` function normally moves to a new line after printing. You may use the `end` parameter to control what is printed after a value.

```
print(1, end=', ')
print(2, end=', ')
print(3)
```

This produces:

```
1, 2, 3
```

Testing
To test your function, type the following command in the terminal:

```
pytest tests/test_q7.py
```

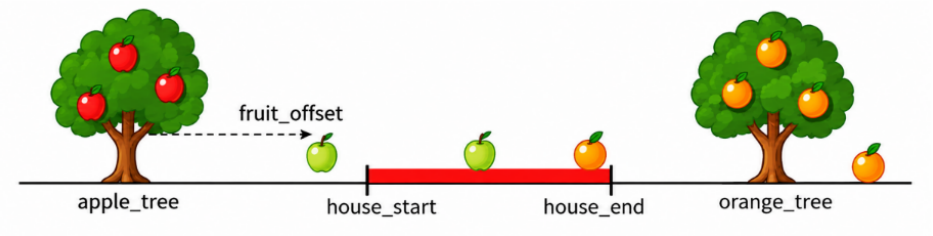
Bonus Exercises

8. Fruits Landing on a House

This question should be implemented in the file `q8.py`.

Mohammad's house lies on a number line from position `house_start` to position `house_end`, inclusive. An apple tree is located at position `apple_tree`, to the left of the house, and an orange tree is located at position `orange_tree`, to the right of the house.

When a fruit falls from a tree, its position relative to the tree is represented by an integer `fruit_offset`. A positive offset means that the fruit lands to the right of the tree, a negative offset means that it lands to the left, and an offset of 0 means that it lands at the tree's position.



For example, if the apple tree is at position 5 and an apple has a `fruit_offset` of -2, then the apple lands at position:

$$5 + (-2) = 3$$

Your task is to determine how many apples and oranges land on Mohammad's house.

Function Specifications

1. Write an iterative function named `apples_in_house` that takes four integer parameters: `house_start`, `house_end`, `apple_tree`, and `total_apples`.
   The function must read exactly `total_apples` integer input values. Each input represents the fruit offset of one apple from the apple tree at position `apple_tree`.
   For each apple, calculate its landing position and count the apple if its landing position lies between `house_start` and `house_end`, inclusive.
   The function must return the number of apples that land on the house.
2. Write an iterative function named `oranges_in_house` that takes four integer parameters: `house_start`, `house_end`, `orange_tree`, and `total_oranges`.
   The function must read exactly `total_oranges` integer input values. Each input represents the fruit offset of one orange from the orange tree at position `orange_tree`.
   For each orange, calculate its landing position and count the orange if its landing position lies between `house_start` and `house_end`, inclusive.
   The function must return the number of oranges that land on the house.
3. Write a main function named `fruits_in_house` that takes six integer parameters: `house_start`, `house_end`, `apple_tree`, `orange_tree`, `total_apples`, and `total_oranges`.
   The function must call `apples_in_house` and `oranges_in_house` to determine the number of apples and oranges that land on the house.
   It must print the two counts on one line, separated by one space. Print the apple count first, followed by the orange count.

Constraints

- $1 \leq$ `house_start` ≤ 1000
- $1 \leq$ `house_end` ≤ 1000
- $1 \leq$ `apple_tree` ≤ 1000
- $1 \leq$ `orange_tree` ≤ 1000
- $1 \leq$ `total_apples` ≤ 1000
- $1 \leq$ `total_oranges` ≤ 1000
- `fruit_offset` is an integer.
- $-1000 \leq$ `fruit_offset` ≤ 1000
- `apple_tree` $<$ `house_start` $<$ `house_end` $<$ `orange_tree`

Interaction
The values `house_start`, `house_end`, `apple_tree`, `orange_tree`, `total_apples`, and `total_oranges` are supplied as arguments when `fruits_in_house` is called.

During the function call:

- `apples_in_house` reads exactly `total_apples` integer fruit offset values, one for each apple;
- `oranges_in_house` then reads exactly `total_oranges` integer fruit offset values, one for each orange.

Therefore, a total of `total_apples + total_oranges` input values are read.

Sample Interaction
In the sample below, lines written as `[Input -> value]` represent values entered by the user while `fruits_in_house` is running. They are not output produced by the function.

```
>>> fruits_in_house(7, 11, 5, 15, 3, 2)
[Input -> -2]
[Input -> 2]
[Input -> 1]
[Input -> 5]
[Input -> -6]
1 1
```

The apple tree is at position 5. The three apples land at:

$$5 + (-2) = 3, \quad 5 + 2 = 7, \quad 5 + 1 = 6$$

Only the apple at position 7 lands on the house.

The orange tree is at position 15. The two oranges land at:

$$15 + 5 = 20, \quad 15 + (-6) = 9$$

Only the orange at position 9 lands on the house. Therefore, one apple and one orange land on the house, so the function prints:

```
1 1
```

Testing
To test your function, type the following command in the terminal:

```
pytest tests/test_q8.py
```

9. Lucky Raffle Ticket

This question should be implemented in the file `q9.py`.

During a university carnival, the raffle organizers use a special rule to decide whether a ticket is lucky. A ticket is considered *Lucky* if its digits can be rearranged to form an integer divisible by 5. Otherwise, the ticket is considered *Not Lucky*.

Function Specification
Write an iterative function named `lucky_ticket` that takes one positive integer parameter, `ticket_number`. The function must determine whether the digits of `ticket_number` can be rearranged to form an integer divisible by 5.

The function must:

- return `"Yes"` if such a rearrangement is possible;
- return `"No"` otherwise.

Constraints

- $1 \leq$ `ticket_number` ≤ 100000
- Solve this question arithmetically using iteration.
- You are not allowed to convert `ticket_number` to a string.

Sample Interaction

```
>>> lucky_ticket(101)
'Yes'
>>> lucky_ticket(119)
'No'
```

For `ticket_number` = 101, its digits can be rearranged to form 110, which is divisible by 5. Therefore, the function returns `"Yes"`.

For `ticket_number` = 119, its digits cannot be rearranged to form an integer divisible by 5, so the function returns `"No"`.

Testing
To test your function, type the following command in the terminal:

```
pytest tests/test_q9.py
```

Rough Work